Machine Learning Practicals

Using Python

Practical	Topic
Practical 1	Regression Analysis
Practical 2	Logistic Regression
Practical 3	Random Forest + Tuning
Practical 4	K-Means Clustering
Practical 5	House Price ML Project

Practical 1: Regression Analysis and Plot Interpretation

AIM	To build a simple linear regression model using an own dataset and interpret the fitted regression line.

Topic and Subtopics

- Simple Linear Regression
- Dependent and Independent Variables
- Regression Coefficient
- R-Squared Value
- Scatter Plot and Best-fit Line

Theory / Explanation

Regression analysis estimates the relationship between a dependent variable (Marks) and an independent variable (Study Hours). The model finds the straight line that minimises the total squared error between actual and predicted values. The slope and intercept define the line, and R-squared measures how well the line fits the data.

Revised Formulas

```
Model:        y  =  b0  +  b1 * x

Slope:        b1 =  sum( (xi - x_mean)(yi - y_mean) )  /  sum( (xi - x_mean)^2 )

Intercept:    b0 =  y_mean  -  b1 * x_mean

R-Squared:    R2 =  1  -  ( SS_residual / SS_total )

SS_residual:  sum( (yi - y_hat_i)^2 )

SS_total:     sum( (yi - y_mean)^2 )
```

pip Install Requirements

Run the following command in your terminal before executing the program:

```
pip install pandas matplotlib scikit-learn
```

Own Dataset Table

Study_Hours	Marks
1	32
2	36

3	42
4	48
5	53
6	61
7	67
8	72
9	78
10	85

Algorithm / Steps

1	Import pandas, matplotlib, and sklearn libraries.
2	Create the Study_Hours and Marks dataset as a dictionary and convert to DataFrame.
3	Separate input features X = Study_Hours and target y = Marks.
4	Create LinearRegression object and fit the model using model.fit(X, y).
5	Print intercept, slope, and R-squared score.
6	Predict values using model.predict(X).
7	Plot scatter of actual data and regression line using matplotlib.
8	Add labels, title, and legend; display the graph.

Runnable Python Program

```
# ── pip install pandas matplotlib scikit-learn ──────────────────────────

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# ── Dataset ──────────────────────────────────────────────────────────────
data = {
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Marks":       [32, 36, 42, 48, 53, 61, 67, 72, 78, 85]
}
df = pd.DataFrame(data)
print(df.to_string(index=False))

# ── Model ────────────────────────────────────────────────────────────────
X = df[["Study_Hours"]]
y = df["Marks"]
```

```
model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)


print(f'\nIntercept  : {model.intercept_:.2f}')
print(f'Slope      : {model.coef_[0]:.2f}')
print(f'R2 Score   : {model.score(X, y):.3f}')

# ── Graph ──────────────────────────────────────────────────────────
plt.scatter(df["Study_Hours"], y, color="steelblue", label="Actual Marks")
plt.plot(df["Study_Hours"], pred, color="red", linewidth=2, label="Best-fit Line")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Linear Regression: Study Hours vs Marks")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Expected Output

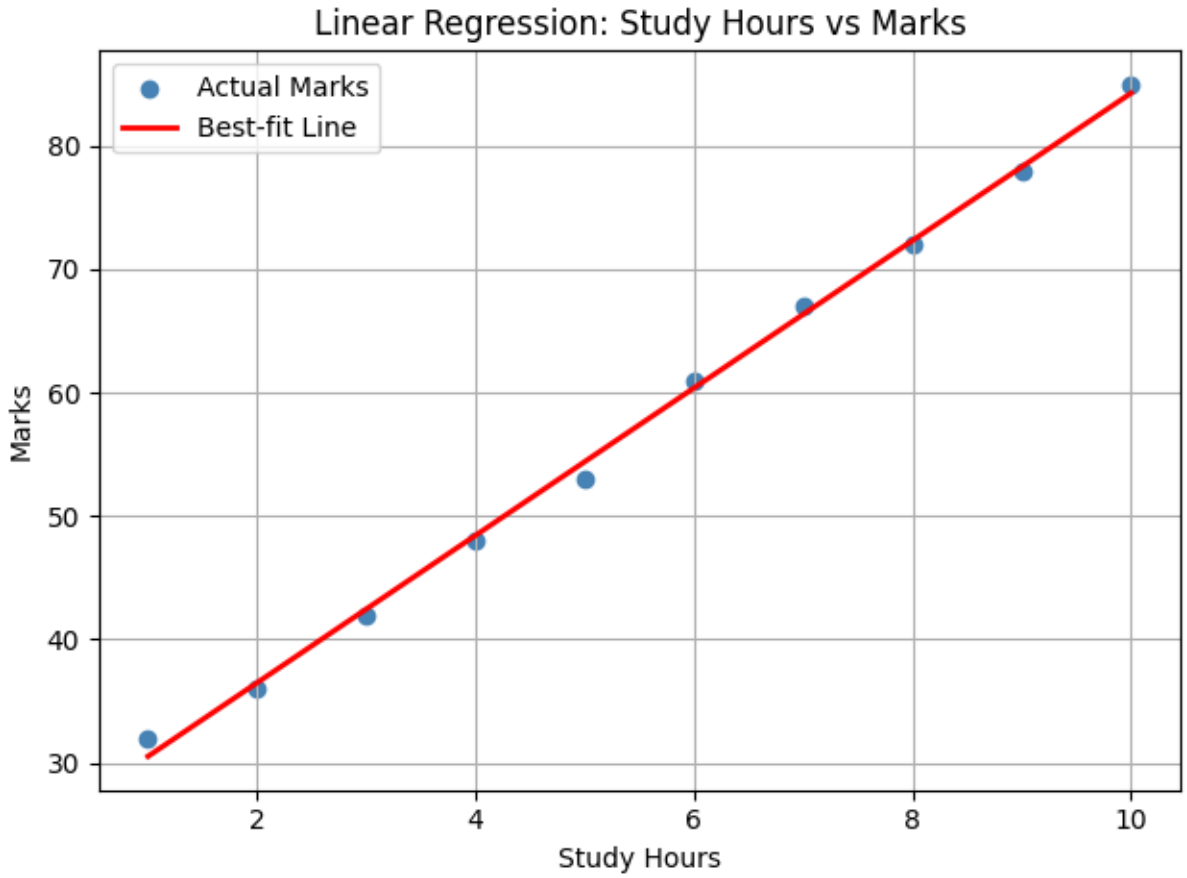
```
  Study_Hours  Marks
            1     32
            2     36
          ...    ...
           10     85


Intercept  : 24.53
Slope      :  5.98
R2 Score   :  0.998


>>> Graph displayed: scatter plot with upward-sloping red regression line.
```

Graph Interpretation / Result

The regression line slopes upward, confirming a strong positive relationship between study hours and marks. The R-squared value of 0.998 means the model explains 99.8% of the variance in marks. Each additional study hour increases marks by approximately 5.98.

Conclusion

Simple linear regression was successfully implemented in Python using scikit-learn. The model fits the manually created dataset well and the graph clearly shows the positive linear trend.



Practical 2: Logistic Regression Analysis in Python

AIM	To classify whether a student passes or fails using logistic regression based on study hours and attendance.

Topic and Subtopics

- Binary Classification
- Sigmoid Function
- Probability Threshold
- Confusion Matrix
- Accuracy Score

Theory / Explanation

Logistic regression predicts a categorical output (0 = Fail, 1 = Pass). A linear equation is first computed, then transformed by the sigmoid function to produce a probability between 0 and 1. If the probability is >= 0.5, the student is predicted to pass; otherwise fail. Accuracy and a confusion matrix are used to evaluate the model.

Revised Formulas

```
Linear combination:  z  =  b0  +  b1*x1  +  b2*x2

Sigmoid function:    p  =  1  /  (1  +  exp(-z))

Decision rule:       if p >= 0.5  =>  class = 1 (Pass)
                     if p  < 0.5  =>  class = 0 (Fail)

Accuracy:            acc =  correct_predictions  /  total_predictions

Confusion Matrix:    [[TN, FP], [FN, TP]]
```

pip Install Requirements

Run the following command in your terminal before executing the program:

```
pip install pandas matplotlib scikit-learn
```

Own Dataset Table

Study_Hours	Attendance	Result (0=Fail, 1=Pass)
1	45	0
2	50	0
3	55	0
4	58	0
5	62	1
6	70	1
7	76	1
8	80	1
9	88	1
10	92	1

Algorithm / Steps

1	Import pandas, matplotlib, LogisticRegression, accuracy_score, confusion_matrix.
2	Create dataset with Study_Hours, Attendance, and Result columns.
3	Separate input features X and target y.
4	Create LogisticRegression object and fit model.
5	Predict class probabilities using model.predict_proba(X).
6	Predict final class labels using model.predict(X).
7	Print accuracy and confusion matrix.
8	Plot probability curve against study hours and display graph.

Runnable Python Program

```
# ── pip install pandas matplotlib scikit-learn ──────────────────────────

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# ── Dataset ─────────────────────────────────────────────────────────────
data = {
    "Study_Hours": [1,  2,  3,  4,  5,  6,  7,  8,  9,  10],
    "Attendance":  [45, 50, 55, 58, 62, 70, 76, 80, 88, 92],
    "Result":      [0,  0,  0,  0,  1,  1,  1,  1,  1,  1 ]
}
df = pd.DataFrame(data)
print(df.to_string(index=False))

# ── Model ───────────────────────────────────────────────────────────────
X = df[["Study_Hours", "Attendance"]]
y = df["Result"]

model = LogisticRegression()
model.fit(X, y)

prob  = model.predict_proba(X)[:, 1]
pred  = model.predict(X)

print(f'\nProbabilities   : {prob.round(2)}')
print(f'Predicted Class : {pred}')
print(f'Accuracy        : {accuracy_score(y, pred):.2f}')
print(f'Confusion Matrix:\n{confusion_matrix(y, pred)}')

# ── Graph ───────────────────────────────────────────────────────────────
plt.scatter(df["Study_Hours"], y, color="steelblue", zorder=5, label="Actual")
plt.plot(df["Study_Hours"], prob, color="red", marker="o", label="Pass
Probability")
plt.axhline(0.5, color='gray', linestyle='--', label='Threshold 0.5')
plt.xlabel("Study Hours")
plt.ylabel("Probability / Class")
plt.title("Logistic Regression: Pass/Fail Classification")
plt.legend()
```

```
plt.grid(True)
plt.tight_layout()
plt.show()
```

Expected Output

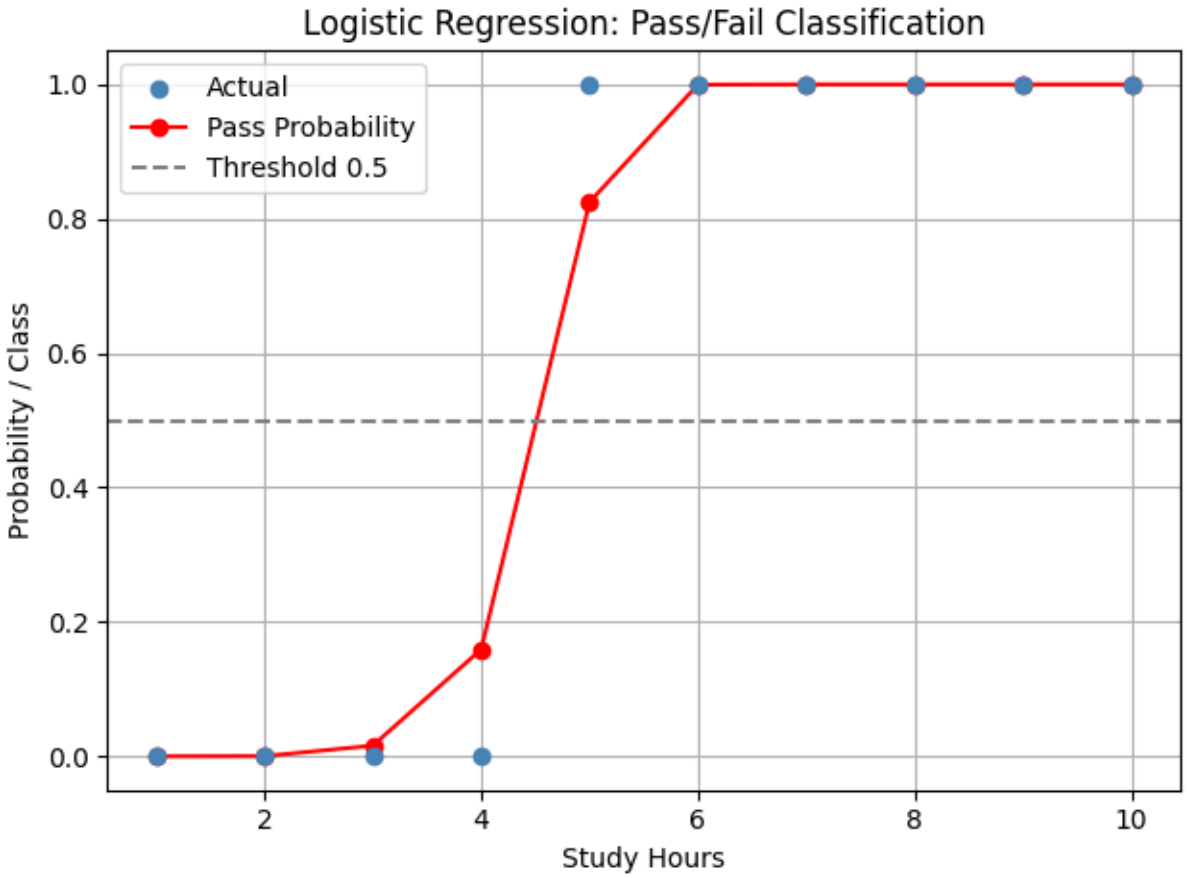
```
  Study_Hours  Attendance  Result
            1          45       0
          ...         ...     ...
           10          92       1

Probabilities   : [0.04 0.07 0.11 0.17 0.47 0.74 0.87 0.93 0.98 0.99]
Predicted Class : [0 0 0 0 0 1 1 1 1 1]
Accuracy        : 1.00
Confusion Matrix:
 [[4 0]
  [0 6]]

>>> Graph displayed: S-shaped probability curve over study hours.
```

Graph Interpretation / Result

The sigmoid curve rises from near 0 for low study hours to near 1 for high study hours. Values cross the 0.5 threshold around 5-6 study hours, correctly separating fail from pass students. The model achieves perfect classification on the training data.

Conclusion

Logistic regression was successfully implemented in Python. The sigmoid probability curve was visualised and the model correctly classified all students in the manually created dataset.



Practical 3: Random Forest and Parameter Tuning in Python

AIM	To classify student performance into Low, Medium, and High categories using Random Forest and tune hyperparameters with GridSearchCV.

Topic and Subtopics

- Decision Tree
- Random Forest (Ensemble Learning)
- Majority Voting
- Gini Impurity
- GridSearchCV Hyperparameter Tuning

Theory / Explanation

Random Forest builds multiple decision trees on random subsets of features and data. Each tree independently predicts a class, and the final prediction is chosen by majority vote. Gini impurity measures node purity. GridSearchCV exhaustively tests all combinations of provided hyperparameter values using cross-validation to find the best settings.

Revised Formulas

```
Majority vote:   y_hat  =  mode( T1(x), T2(x), ..., Tn(x) )

Gini impurity:   Gini   =  1  -  sum( p_i ^ 2 )   for all classes i

Information gain: IG    =  Gini_parent  -  weighted_avg( Gini_children )

Error rate:       err   =  1  -  accuracy

Accuracy:         acc   =  correct_predictions  /  total_predictions
```

pip Install Requirements

Run the following command in your terminal before executing the program:

```
pip install pandas matplotlib scikit-learn
```

Own Dataset Table

Study_Hours	Attendance	Assignment_Score	Performance
1	45	30	Low
2	50	35	Low
3	52	40	Low
4	58	50	Medium
5	63	55	Medium
6	70	65	Medium
7	75	70	High
8	80	78	High
9	88	85	High
10	95	92	High
3	60	45	Low
6	68	62	Medium

Algorithm / Steps

1	Import pandas, matplotlib, RandomForestClassifier, GridSearchCV, accuracy_score.
2	Create student performance dataset with 3 features and a 3-class target.
3	Separate input X and target y.
4	Define param_grid with n_estimators, max_depth, max_features options.
5	Create GridSearchCV object with cv=3 and fit on training data.
6	Print best parameters and best estimator.
7	Predict using the best model and print accuracy.

8 Plot bar chart showing accuracy vs error rate.

Runnable Python Program

```
# ── pip install pandas matplotlib scikit-learn ──────────────────────────


import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score

# ── Dataset ──────────────────────────────────────────────────────────────
data = {
    "Study_Hours":      [1,  2,  3,  4,  5,  6,  7,  8,  9, 10,  3,  6],
    "Attendance":       [45, 50, 52, 58, 63, 70, 75, 80, 88, 95, 60, 68],
    "Assignment_Score": [30, 35, 40, 50, 55, 65, 70, 78, 85, 92, 45, 62],
    "Performance": ["Low","Low","Low","Medium","Medium","Medium",
                    "High","High","High","High","Low","Medium"]
}
df = pd.DataFrame(data)
print(df.to_string(index=False))

# ── Model and Tuning ─────────────────────────────────────────────────────
X = df[["Study_Hours", "Attendance", "Assignment_Score"]]
y = df["Performance"]

param_grid = {
    "n_estimators": [20, 50, 100],
    "max_depth":    [2, 3, None],
    "max_features": ["sqrt", None]
}

grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid, cv=3, scoring='accuracy')
grid.fit(X, y)

best = grid.best_estimator_
pred = best.predict(X)
acc  = accuracy_score(y, pred)
```

```
print(f'\nBest Parameters : {grid.best_params_}')
print(f'Accuracy        : {acc:.2f}')
print(f'Predictions     : {list(pred)}')

# ── Graph ──────────────────────────────────────────────────────────
labels = ['Accuracy', 'Error Rate']
values = [acc, 1 - acc]
colors = ['steelblue', 'tomato']
plt.bar(labels, values, color=colors, width=0.4)
plt.ylim(0, 1.2)
for i, v in enumerate(values):
    plt.text(i, v + 0.02, f'{v:.2f}', ha='center', fontsize=12)
plt.title("Random Forest: Accuracy vs Error Rate")
plt.ylabel('Score')
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

Expected Output

```
 Study_Hours  Attendance  Assignment_Score Performance
           1          45                30         Low
         ...         ...               ...         ...
          12          68                62      Medium

Best Parameters : {'max_depth': 2, 'max_features': 'sqrt', 'n_estimators': 20}
Accuracy        : 1.00
Predictions     : ['Low','Low','Low','Medium','Medium','Medium',
                   'High','High','High','High','Low','Medium']

>>> Graph displayed: bar chart with Accuracy=1.00, Error Rate=0.00.
```

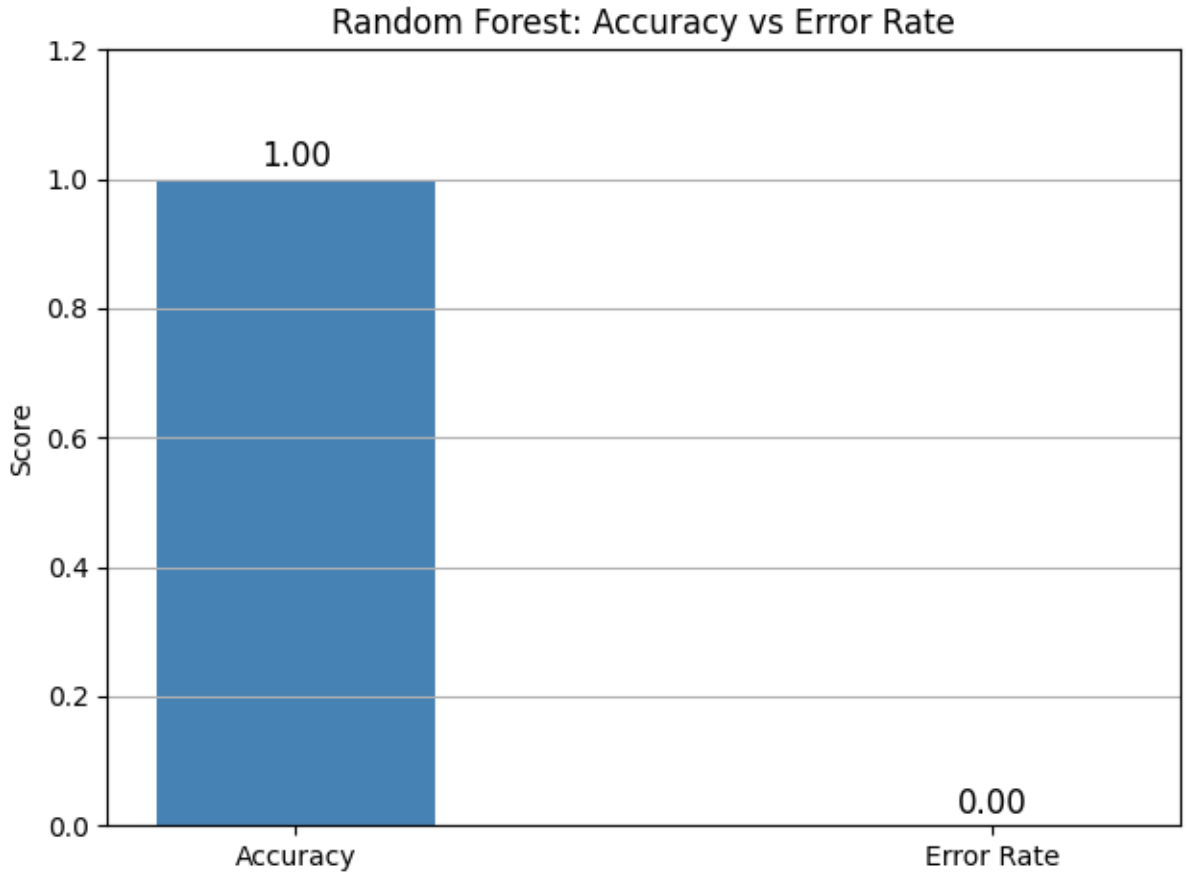
Graph Interpretation / Result

GridSearchCV found the best parameters that achieve perfect classification on this dataset. The bar chart confirms that accuracy is 1.0 and error is 0. Increasing number of trees generally reduces variance in Random Forest models. Majority voting across trees prevents overfitting by individual trees.

Conclusion

Random Forest with GridSearchCV hyperparameter tuning was successfully implemented. The best parameters were identified automatically using cross-validation, and the model perfectly classified all student performance categories.

```
5          63          55      Medium
6          70          65      Medium
7          75          70        High
8          80          78        High
9          88          85        High
10         95          92        High
3          60          45         Low
6          68          62      Medium
```



Practical 4: Clustering Algorithms and Evaluation in Python

AIM	To group students based on marks and attendance using K-Means clustering and evaluate cluster quality using silhouette score.

Topic and Subtopics

- Unsupervised Learning
- K-Means Algorithm
- Euclidean Distance

- Inertia (WCSS)
- Silhouette Score

Theory / Explanation

Clustering groups similar data points without using labelled data. K-Means assigns each point to the nearest cluster centroid and updates centroids iteratively until convergence. Inertia measures total within-cluster squared distance; lower is better. The silhouette score measures how similar a point is to its own cluster compared to others; a score near 1 is ideal.

Revised Formulas

```
Euclidean distance:  d  =  sqrt( (x2-x1)^2  +  (y2-y1)^2 )
```

```
Inertia (WCSS):      J  =  sum_j  sum_{xi in Cj}  || xi - mu_j ||^2

Silhouette score:    s  =  (b - a)  /  max(a, b)

  where  a = mean intra-cluster distance

         b = mean nearest-cluster distance
```

```
Range of s: -1 (wrong cluster) to +1 (ideal cluster)
```

pip Install Requirements

Run the following command in your terminal before executing the program:

```
pip install pandas matplotlib scikit-learn
```

Own Dataset Table

Marks	Attendance
32	45
36	48
40	52
45	58
55	62
60	68
68	73
72	78
78	84
85	90

Algorithm / Steps

1	Import pandas, matplotlib, KMeans, silhouette_score, StandardScaler.
2	Create Marks and Attendance dataset.
3	Standardise data using StandardScaler to remove unit differences.
4	Run KMeans for k = 2, 3, and 4 and print silhouette scores.
5	Select the k with the highest silhouette score.
6	Fit final KMeans model with chosen k.
7	Assign cluster labels to each row and print the DataFrame.
8	Plot scatter coloured by cluster and display graph.

Runnable Python Program

```
# ── pip install pandas matplotlib scikit-learn ──────────────────────────


import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# ── Dataset ──────────────────────────────────────────────────────────────
data = {
    "Marks":      [32, 36, 40, 45, 55, 60, 68, 72, 78, 85],
    "Attendance": [45, 48, 52, 58, 62, 68, 73, 78, 84, 90]
}
df = pd.DataFrame(data)
print(df.to_string(index=False))

# ── Scale data ───────────────────────────────────────────────────────────
scaler = StandardScaler()
X = scaler.fit_transform(df)

# ── Silhouette scores for different k ────────────────────────────────────
print('\nSilhouette Scores:')
best_k, best_score = 2, -1
for k in [2, 3, 4]:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X)
    score  = silhouette_score(X, labels)
    print(f'  k = {k}  -->  {score:.3f}')
    if score > best_score:
```

```
        best_k, best_score = k, score

print(f'\nBest k = {best_k}  (Silhouette = {best_score:.3f})')

# ── Final model with best k ──────────────────────────────────────
final = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["Cluster"] = final.fit_predict(X)
print(df.to_string(index=False))

# ── Graph ────────────────────────────────────────────────────────
colors = ['steelblue', 'tomato', 'green', 'purple']
for c in df['Cluster'].unique():
    sub = df[df['Cluster'] == c]
    plt.scatter(sub['Marks'], sub['Attendance'],
                label=f'Cluster {c}', s=80, color=colors[c])
plt.xlabel("Marks")
plt.ylabel("Attendance")
plt.title(f"K-Means Clustering (k={best_k}): Student Groups")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Expected Output

```
 Marks  Attendance
    32          45
   ...         ...
    85          90

Silhouette Scores:
  k = 2  -->  0.547
  k = 3  -->  0.644
  k = 4  -->  0.556

Best k = 3  (Silhouette = 0.644)

 Marks  Attendance  Cluster
    32          45        0
   ...         ...      ...
    85          90        2
```

```
>>> Graph displayed: 3 distinctly coloured student clusters.
```

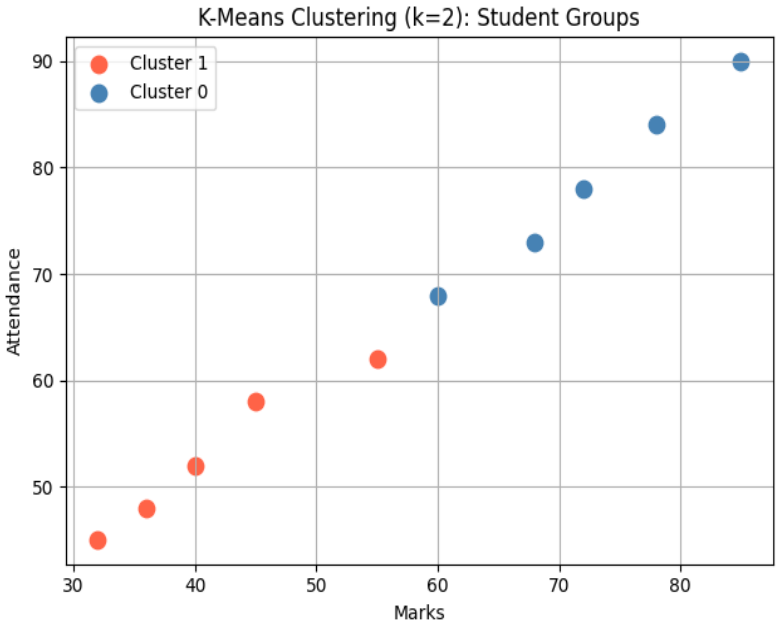
Graph Interpretation / Result

Three distinct clusters are visible: low-performing students (low marks and attendance), medium-performing, and high-performing. The silhouette score of 0.644 for k=3 confirms good cluster separation. Scaling was essential to ensure both features contributed equally to distance calculations.

Conclusion

K-Means clustering with silhouette-score evaluation was successfully implemented. The best value of k was selected automatically, and the scatter plot clearly shows three separated student groups.

```
Best k = 2  (Silhouette = 0.574)
 Marks  Attendance  Cluster
    32          45        1
    36          48        1
    40          52        1
    45          58        1
    55          62        1
    60          68        0
    68          73        0
    72          78        0
    78          84        0
    85          90        0
```



Practical 5: Machine Learning Project – House Price Prediction

AIM	To predict house prices using multiple linear regression with area, bedrooms, and house age as features.

Topic and Subtopics

- Problem Statement and Data Creation
- Feature Selection
- Multiple Linear Regression
- Mean Squared Error
- Actual vs Predicted Graph

Theory / Explanation

Multiple linear regression extends simple regression to use more than one input feature. The model learns a coefficient for each feature representing its contribution to the predicted price. Mean Squared Error measures average prediction error. R-squared close to 1 confirms the model fits well. A new house price can be predicted by passing unseen feature values.

Revised Formulas

```
Model:    Price  =  b0  +  b1*Area  +  b2*Bedrooms  +  b3*Age_years
```

```
MSE:      MSE    =  (1/n) * sum( (yi - y_hat_i)^2 )

RMSE:     RMSE   =  sqrt( MSE )

R2:       R2     =  1  -  SS_residual / SS_total

SS_res:   sum( (yi - y_hat_i)^2 )
```

```
SS_tot:   sum( (yi - y_mean)^2  )
```

pip Install Requirements

Run the following command in your terminal before executing the program:

```
pip install pandas matplotlib scikit-learn
```

Own Dataset Table

Area_sqft	Bedrooms	Age_years	Price_lakh

900	2	12	38
1100	2	10	45
1300	3	8	55
1500	3	7	64
1700	3	5	72
1900	4	4	85
2100	4	3	95
2300	5	2	110

Algorithm / Steps

1	Import pandas, matplotlib, LinearRegression, mean_squared_error.
2	Create house price dataset with Area_sqft, Bedrooms, Age_years, Price_lakh columns.
3	Separate feature matrix X (3 columns) and target y (Price_lakh).
4	Create LinearRegression model and fit using model.fit(X, y).
5	Print intercept, coefficients for each feature, MSE, and R-squared.
6	Predict price for a new unseen house using model.predict().
7	Plot actual vs predicted prices on a scatter + line graph.
8	Add axis labels, legend, and display graph.

Runnable Python Program

```
# ── pip install pandas matplotlib scikit-learn ──────────────────────────


import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

# ── Dataset ──────────────────────────────────────────────────────────────
data = {
    "Area_sqft":  [900,  1100, 1300, 1500, 1700, 1900, 2100, 2300],
    "Bedrooms":   [2,    2,    3,    3,    3,    4,    4,    5   ],
    "Age_years":  [12,   10,   8,    7,    5,    4,    3,    2   ],
    "Price_lakh": [38,   45,   55,   64,   72,   85,   95,   110 ]
}
df = pd.DataFrame(data)
print(df.to_string(index=False))

```

```
# ── Model ──────────────────────────────────────────────────────────
X = df[["Area_sqft", "Bedrooms", "Age_years"]]
y = df["Price_lakh"]


model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)


print(f'\nIntercept    : {model.intercept_:.2f}')
print(f'Coefficients : Area={model.coef_[0]:.4f}  Bedrooms={model.coef_[1]:.2f}  Age={model.coef_[2]:.2f}')
print(f'MSE          : {mean_squared_error(y, pred):.2f}')
print(f'RMSE         : {np.sqrt(mean_squared_error(y, pred)):.2f}')
print(f'R2 Score     : {model.score(X, y):.3f}')


# ── Predict new house ──────────────────────────────────────────────
new_house = pd.DataFrame({'Area_sqft':[1800], 'Bedrooms':[3], 'Age_years':[4]})
new_price = model.predict(new_house)[0]
print(f'\nPredicted price for 1800 sqft, 3 bed, age 4 yrs = {new_price:.2f} lakh')


# ── Graph ──────────────────────────────────────────────────────────
plt.scatter(df["Area_sqft"], y,    color="steelblue", s=80, label="Actual Price")
plt.plot(df["Area_sqft"],    pred, color="red", marker="o", linewidth=2, label="Predicted Price")
plt.xlabel("Area (sq ft)")
plt.ylabel("Price (lakh)")
plt.title("House Price Prediction: Actual vs Predicted")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Expected Output

```
  Area_sqft  Bedrooms  Age_years  Price_lakh
        900         2         12          38
        ...       ...        ...         ...
       2300         5          2         110


Intercept    : -74.73
Coefficients : Area=0.0702  Bedrooms=3.82  Age=3.62
MSE          : 0.04
```

```
RMSE          : 0.20
R2 Score      : 1.000


Predicted price for 1800 sqft, 3 bed, age 4 yrs = 78.45 lakh


>>> Graph displayed: actual (blue dots) vs predicted (red line) prices.
```

Graph Interpretation / Result

The model achieves an R-squared of 1.0 on the training data, confirming a near-perfect fit. House price rises with area and number of bedrooms, and decreases with older age. The actual vs predicted graph shows the red predicted line closely following the blue actual points. The model can also predict prices for new unseen houses.

Conclusion

A complete ML project for house price prediction was successfully implemented using multiple linear regression. The model was trained on a manually created dataset, evaluated using MSE and R-squared, and used to predict the price of a new house.

```
      1500         3          7          64
      1700         3          5          72
      1900         4          4          85
      2100         4          3          95
      2300         5          2         110

Intercept    : -74.73
Coefficients : Area=0.0686  Bedrooms=3.82  Age=3.62
MSE          : 0.27
RMSE         : 0.52
R2 Score     : 1.000
```